

POSTGRESQL

MVCC & BLOAT

Why Bloat Does Not Automatically Mean Expensive Reads

END-TO-END TECHNICAL SOP & HANDS-ON LAB
DVD Rental Sample Database | PostgreSQL 18

MVCC • Tuple Versions • HOT • Dead Tuples • Heap Pruning • VACUUM • Visibility Map • Index-Only Scans •
Bloat Diagnostics

Validated against PostgreSQL 18 official documentation

Target audience: Senior DBA / Database Engineer / Production Support / Architect

Lab database: dvdrental

Document date: 10 September 2026

Contents

1. Executive Summary
2. Scope, Objectives and Safety
3. PostgreSQL MVCC Fundamentals
4. How UPDATE and DELETE Create Tuple Versions
5. HOT Updates and Same-Page Optimization
6. Bloat: Storage Amplification vs Read Amplification
7. DVD Rental Lab Architecture
8. Lab Prerequisites
9. Build the mvcc_lab Environment
10. Baseline Table and Index Measurements
11. Inspect xmin, xmax and ctid
12. HOT Update Demonstration
13. Non-HOT Update Demonstration
14. Generate Controlled MVCC Churn
15. Measure Dead Tuples and Physical Space
16. Sequential Scan Behavior
17. Index Scan Behavior
18. Visibility Map
19. Index-Only Scan Lab
20. Heap Pruning and HOT Chains
21. VACUUM and Autovacuum
22. VACUUM FULL
23. DELETE and Transaction Visibility
24. Long-Running Transactions and Cleanup Delay
25. Autovacuum Tuning for High-Churn Tables
26. Production Troubleshooting Workflow
27. Monitoring Queries
28. Common Myths and Corrections
29. Real-Time Production Scenario
30. DBA Decision Matrix
31. End-to-End Lab Checklist
32. Cleanup
33. Final Technical Conclusions
34. PostgreSQL Official Documentation References

1. Executive Summary

PostgreSQL uses Multiversion Concurrency Control (MVCC) so readers and writers can work concurrently while each transaction sees a consistent snapshot. An UPDATE normally creates a new heap tuple version, while an old version remains until PostgreSQL can safely reclaim it. DELETE similarly leaves a tuple that may remain physically present until cleanup.

This behavior can produce dead tuples and relation growth commonly described as bloat. However, the presence of bloat does not prove that every read must walk every historical row version. Query cost depends on the physical number of pages, tuple density, access path, cache state, visibility, index state, long-running transactions and the actual execution plan.

Core principle: BLOAT IS A CAPACITY/PERFORMANCE SIGNAL — NOT PROOF THAT EVERY READ IS EXPENSIVE.

2. Scope, Objectives and Safety

- Use the standard dvdrental sample database as the source data set.
- Create a separate mvcc_lab schema so the original public.rental table is not modified.
- Observe tuple versioning using xmin, xmax and ctid.
- Compare HOT and non-HOT updates.
- Generate controlled dead tuples and measure relation growth.
- Compare sequential, index and index-only scans with EXPLAIN (ANALYZE, BUFFERS).
- Inspect the visibility map with pg_visibility.
- Measure tuple-level space with pgstattuple.
- Demonstrate VACUUM behavior and explain why normal VACUUM usually does not shrink the file.
- Use VACUUM FULL only inside the isolated lab because it rewrites the relation and requires an ACCESS EXCLUSIVE lock.

Production safety: Do not disable autovacuum globally and do not run VACUUM FULL on a busy production table merely because n_dead_tup looks high.

3. PostgreSQL MVCC Fundamentals

MVCC lets different transactions see different valid versions of the same logical row. PostgreSQL stores transaction metadata in tuple headers and evaluates visibility against the reader's transaction snapshot.

Field	Meaning	DBA Note
xmin	Transaction ID that created the tuple version	Useful for understanding tuple creation.
xmax	Used when a tuple is deleted/superseded or locked	Raw interpretation requires tuple-header context; do not treat xmax=0/other values as a complete visibility test.
ctid	Physical tuple location as (block,item)	Can change after UPDATE or table rewrites; never use as a permanent business key.
Logical row ≠ one permanent physical tuple. A logical row may have multiple physical tuple versions over time.		

4. How UPDATE and DELETE Create Tuple Versions

```
BEGIN;
UPDATE public.rental
SET return_date = CURRENT_TIMESTAMP
WHERE rental_id = 100;
COMMIT;
```

Conceptually, an UPDATE creates a newer row version rather than overwriting the old tuple in place.

Original tuple → UPDATE → New tuple version
Old snapshot may still need old tuple | Newer snapshot can see new tuple

DELETE also follows MVCC rules. The tuple is not necessarily erased from its heap page immediately. After the deleting transaction commits and no required snapshot can still see the tuple, VACUUM or related page-level cleanup can reclaim the space.

5. HOT Updates and Same-Page Optimization

Heap-Only Tuple (HOT) is a PostgreSQL optimization for UPDATE workloads. According to PostgreSQL 18 documentation, HOT can be used when the update does not change columns referenced by the table's indexes (excluding summarizing indexes such as BRIN) and there is enough free space on the same heap page.

- New index entries are avoided for the HOT update.
- Indexes continue to identify the original page item; PostgreSQL can follow the HOT chain on that page.
- Intermediate obsolete HOT versions can be removed during normal operation, including SELECT activity, when they are no longer visible to any transaction.
- Lower fillfactor can increase the chance that a page retains room for updated tuples.
- HOT still creates a new heap tuple version; it is not an in-place row overwrite.

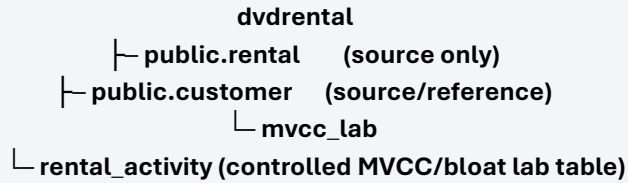
UPDATE unindexed column + same-page free space → HOT may be possible
UPDATE indexed column → regular update; index maintenance is required

6. Bloat: Storage Amplification vs Read Amplification

Concept	Meaning	Example
Storage amplification	Physical storage exceeds the logical useful data footprint	20 MB useful data occupying 80 MB of heap/index storage.
Read amplification	A query must perform materially more page/tuple work to return the requested rows	A sequential scan touches 40,000 pages instead of 10,000 after heavy relation growth.

These concepts are related but are not identical. A bloated table can slow reads because it may contain more heap pages, consume more cache, and require more I/O. But this is different from claiming that each `SELECT` must traverse all historical versions of every row.

7. DVD Rental Lab Architecture



8. Lab Prerequisites

- PostgreSQL 18 server with the dvdrental sample database restored.
- Privileges to create schema, tables, indexes and supplied extensions in the lab database.
- psql or a SQL client such as pgAdmin.
- Enough free disk space for repeated updates and a controlled VACUUM FULL test.
- Run VACUUM FULL only during the isolated test; it is not required for the normal sections.

```
SELECT current_database(),
       current_user,
       version();
```

9. Build the mvcc_lab Environment

```
DROP SCHEMA IF EXISTS mvcc_lab CASCADE;
CREATE SCHEMA mvcc_lab;

CREATE EXTENSION IF NOT EXISTS pgstattuple;
CREATE EXTENSION IF NOT EXISTS pg_visibility;
CREATE EXTENSION IF NOT EXISTS pageinspect;

CREATE TABLE mvcc_lab.rental_activity AS
SELECT
    r.rental_id,
    r.customer_id,
    r.inventory_id,
    r.rental_date,
    r.return_date,
    repeat('X', 800)::text AS notes
FROM public.rental r;

ALTER TABLE mvcc_lab.rental_activity
ADD CONSTRAINT pk_rental_activity
PRIMARY KEY (rental_id);

CREATE INDEX ix_rental_activity_customer
ON mvcc_lab.rental_activity(customer_id);

CREATE INDEX ix_rental_activity_return_date
ON mvcc_lab.rental_activity(return_date);

ANALYZE mvcc_lab.rental_activity;
```

pgstattuple, pg_visibility and pageinspect are PostgreSQL-supplied modules. pageinspect exposes low-level page data and should be used as an advanced DBA diagnostic tool.

10. Baseline Table and Index Measurements

```
SELECT count(*) AS row_count
FROM mvcc_lab.rental_activity;
```

```
SELECT
    pg_size_pretty(pg_relation_size('mvcc_lab.rental_activity')) AS heap_size,
    pg_size_pretty(pg_indexes_size('mvcc_lab.rental_activity')) AS index_size,
    pg_size_pretty(pg_total_relation_size('mvcc_lab.rental_activity')) AS total_size;
```

```
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    n_tup_ins,
    n_tup_upd,
    n_tup_del,
    n_tup_hot_upd,
    last_vacuum,
    last_autovacuum,
    last_analyze,
    last_autoanalyze
FROM pg_stat_user_tables
WHERE schemaname = 'mvcc_lab'
    AND relname = 'rental_activity';
```

Checkpoint: Save the initial row count, heap size, index size and pg_stat_user_tables values before generating churn.

11. Inspect xmin, xmax and ctid

```
SELECT
    ctid,
    xmin,
    xmax,
    rental_id,
    customer_id,
    return_date
FROM mvcc_lab.rental_activity
ORDER BY rental_id
LIMIT 10;
```

The transaction IDs and CTIDs are system-specific. Do not compare them to fixed sample values. The purpose is to observe that an UPDATE can produce a new physical tuple location and a new creating transaction ID.

12. HOT Update Demonstration

```
SELECT ctid, xmin, xmax, rental_id, notes
FROM mvcc_lab.rental_activity
WHERE rental_id = 1;

UPDATE mvcc_lab.rental_activity
SET notes = notes || '-UPDATED'
WHERE rental_id = 1;

SELECT ctid, xmin, xmax, rental_id, notes
FROM mvcc_lab.rental_activity
WHERE rental_id = 1;
```

The notes column is not indexed. This makes the update eligible for HOT if the page has enough suitable free space. Eligibility does not guarantee HOT; page space and storage conditions still matter.

```
ALTER TABLE mvcc_lab.rental_activity
SET (fillfactor = 80);
```

Changing fillfactor affects future page usage but does not immediately rewrite existing table pages. To compare fillfactor scientifically, recreate or rewrite the test table and repeat the workload under the new setting.

13. Non-HOT Update Demonstration

```
UPDATE mvcc_lab.rental_activity
SET customer_id = customer_id + 10000
WHERE rental_id = 1;
```

customer_id is indexed by ix_rental_activity_customer. Updating it prevents the normal HOT optimization for this row version because index maintenance is needed.

```
SELECT
    n_tup_upd,
    n_tup_hot_upd,
    n_tup_newpage_upd
FROM pg_stat_user_tables
WHERE schemaname = 'mvcc_lab'
    AND relname = 'rental_activity';
```

14. Generate Controlled MVCC Churn

```
DO $$
BEGIN
    FOR i IN 1..100 LOOP
        UPDATE mvcc_lab.rental_activity
        SET notes = 'UPDATE-' || i || '-' || repeat('X', 700)
        WHERE rental_id BETWEEN 1 AND 500;
    END LOOP;
END $$;
```

This workload is intentionally aggressive. Use it only against the lab copy, not the original DVD Rental

tables.

15. Measure Dead Tuples and Physical Space

```
SELECT
    n_live_tup,
    n_dead_tup,
    n_tup_upd,
    n_tup_hot_upd
FROM pg_stat_user_tables
WHERE schemaname = 'mvcc_lab'
    AND relname = 'rental_activity';

SELECT *
FROM pgstattuple('mvcc_lab.rental_activity');

SELECT
    pg_size_pretty(pg_relation_size('mvcc_lab.rental_activity')) AS heap_size,
    pg_size_pretty(pg_indexes_size('mvcc_lab.rental_activity')) AS index_size,
    pg_size_pretty(pg_total_relation_size('mvcc_lab.rental_activity')) AS total_size;
```

`n_live_tup` and `n_dead_tup` in statistics views are estimates. `pgstattuple` performs a more direct physical inspection of tuple and free-space characteristics. Use `pgstattuple` carefully on large busy production relations because detailed inspection can be more expensive than statistics-view checks.

16. Sequential Scan Behavior

```
EXPLAIN
(
    ANALYZE,
    BUFFERS,
    WAL,
    VERBOSE
)
SELECT *
FROM mvcc_lab.rental_activity;
```

Sequential Scan → read heap pages → test tuple visibility → return visible tuples / skip invisible tuples

A sequential scan processes the heap pages and applies MVCC visibility rules. It does not begin with the oldest historical version of each logical row and chase an unlimited chain across the table. However, if bloat expands the physical relation from 10,000 pages to 40,000 pages, the sequential scan may have substantially more page work. That is real read amplification caused by relation growth.

17. Index Scan Behavior

```
EXPLAIN
(
    ANALYZE,
    BUFFERS
```

```

)
SELECT *
FROM mvcc_lab.rental_activity
WHERE customer_id = 100;

```

B-tree index → candidate TID → heap tuple → MVCC visibility check → return/reject

PostgreSQL index access methods return tuple identifiers for matching entries. The heap tuple is normally fetched to perform the visibility test because standard indexes do not carry enough MVCC visibility state for that decision.

18. Visibility Map

Every heap relation has a visibility map (VM). PostgreSQL 18 stores two bits per heap page: all-visible and all-frozen. The map is conservative; data modifications clear relevant bits, while VACUUM sets them when it can prove the condition.

```

SELECT *
FROM pg_visibility_map('mvcc_lab.rental_activity'::regclass)
LIMIT 20;

SELECT
    count(*) AS total_pages,
    count(*) FILTER (WHERE all_visible) AS all_visible_pages,
    count(*) FILTER (WHERE all_frozen) AS all_frozen_pages
FROM pg_visibility_map('mvcc_lab.rental_activity'::regclass);

```

Visibility Map → helps VACUUM skip suitable pages + allows index-only scans to avoid heap access on all-visible pages.

19. Index-Only Scan Lab

```

CREATE INDEX ix_rental_activity_customer_cover
ON mvcc_lab.rental_activity(customer_id)
INCLUDE (rental_id, rental_date, return_date);

VACUUM (ANALYZE) mvcc_lab.rental_activity;

EXPLAIN
(
    ANALYZE,
    BUFFERS
)
SELECT
    rental_id,
    customer_id,
    rental_date,
    return_date
FROM mvcc_lab.rental_activity
WHERE customer_id = 100;

```

If PostgreSQL chooses an Index Only Scan, inspect the 'Heap Fetches' line. Low or zero heap fetches indicate that visibility-map information allowed many or all qualifying rows to be returned without reading their heap pages.

```
UPDATE mvcc_lab.rental_activity
SET notes = notes || '-NEW'
WHERE customer_id = 100;

-- Run the same EXPLAIN again, then:
VACUUM (ANALYZE) mvcc_lab.rental_activity;

-- Repeat EXPLAIN and compare Heap Fetches.
```

20. Heap Pruning and HOT Chains

PostgreSQL can perform page-level pruning during normal operation. HOT is especially important here: when multiple same-page HOT versions exist, intermediate versions that are no longer visible to any transaction can be removed during normal operation, including SELECT activity.

Index entry → original page item → HOT redirect/chain → newest version visible to the snapshot
 Obsolete intermediate HOT versions can be pruned when safe.

This optimization does not make VACUUM optional. VACUUM performs broader dead-tuple processing, index cleanup, visibility-map maintenance and anti-wraparound work.

21. VACUUM and Autovacuum

```
VACUUM (VERBOSE, ANALYZE)
mvcc_lab.rental_activity;

SELECT *
FROM pgstattuple('mvcc_lab.rental_activity');
```

Normal VACUUM makes dead tuple space available for PostgreSQL to reuse and performs associated maintenance. It normally does not compact the entire relation and return most internal free space to the operating system.

Before normal VACUUM	After normal VACUUM
[Live][Dead][Dead][Live][Dead]	[Live][Free][Free][Live][Free]
Physical file can remain large	Space is available for future PostgreSQL reuse

```
SHOW autovacuum;
SHOW autovacuum_vacuum_threshold;
SHOW autovacuum_vacuum_scale_factor;
SHOW autovacuum_analyze_threshold;
SHOW autovacuum_analyze_scale_factor;
SHOW autovacuum_max_workers;
SHOW autovacuum_naptime;
```

22. VACUUM FULL

```
VACUUM FULL mvcc_lab.rental_activity;
```

VACUUM FULL rewrites the relation into a compact physical file and can return space to the operating system. PostgreSQL documentation states that it requires an ACCESS EXCLUSIVE lock on the table while it runs. It also needs additional disk space during the rewrite.

Operation	Makes dead space reusable	Usually shrinks physical file substantially	Locking profile
Autovacuum	Yes	No	Designed for routine online maintenance
VACUUM	Yes	No	Routine maintenance; does not take ACCESS EXCLUSIVE for entire operation
VACUUM (ANALYZE)	Yes	No	Also refreshes planner statistics
VACUUM FULL	Yes	Yes	ACCESS EXCLUSIVE; table rewrite

23. DELETE and Transaction Visibility

```
BEGIN;

DELETE FROM mvcc_lab.rental_activity
WHERE rental_id = 500;

-- Inspect from another session as appropriate for the isolation level.

ROLLBACK;
```

Because DELETE is transactional, a rollback preserves the logical row. Once a DELETE commits, the old tuple can remain physically present until no required snapshot needs it and cleanup can reclaim the space.

24. Long-Running Transactions and Cleanup Delay

```
SELECT
    pid,
    username,
    application_name,
    state,
    xact_start,
    query_start,
    backend_xmin,
    wait_event_type,
    wait_event,
    query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

Long-running transactions can retain old snapshots or horizons that delay removal of obsolete tuple versions. A table may therefore keep accumulating dead tuples even when autovacuum is active.

High `n_dead_tup` + old `xact_start`/`backend_xmin` → investigate long-running/idle-in-transaction sessions before blaming autovacuum alone.

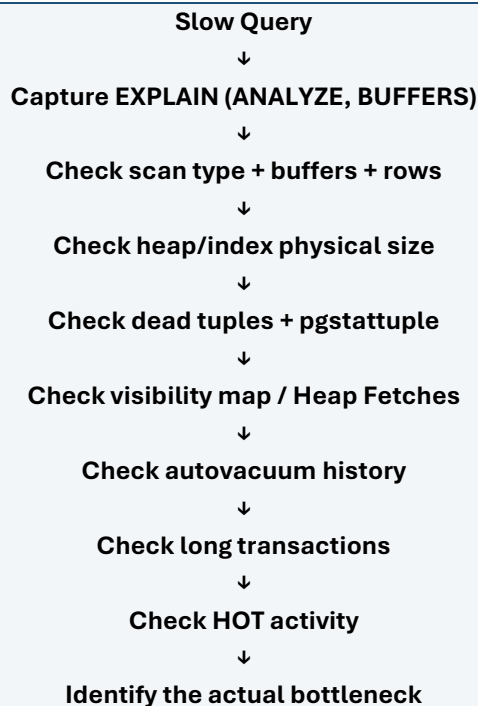
25. Autovacuum Tuning for High-Churn Tables

```
ALTER TABLE mvcc_lab.rental_activity
SET (
    autovacuum_vacuum_scale_factor = 0.05,
    autovacuum_analyze_scale_factor = 0.05
);
```

These are lab examples, not universal production values. Large high-churn relations often benefit from table-specific settings because a percentage-based trigger can represent a very large number of changed tuples.

- Measure update/delete rate.
- Track dead-tuple growth over time.
- Observe autovacuum duration and frequency.
- Check whether workers keep up during peak load.
- Consider storage throughput and I/O contention.
- Use per-table settings for exceptional high-churn objects rather than weakening global defaults without evidence.

26. Production Troubleshooting Workflow



27. Monitoring Queries

```
SELECT
    s.schemaname,
    s.relname,
    s.n_live_tup,
```

```

s.n_dead_tup,
s.n_tup_ins,
s.n_tup_upd,
s.n_tup_del,
s.n_tup_hot_upd,
pg_size_pretty(pg_relation_size(s.relid)) AS table_size,
pg_size_pretty(pg_indexes_size(s.relid)) AS indexes_size,
pg_size_pretty(pg_total_relation_size(s.relid)) AS total_size,
s.last_vacuum,
s.last_autovacuum,
s.last_analyze,
s.last_autoanalyze
FROM pg_stat_user_tables s
ORDER BY pg_total_relation_size(s.relid) DESC;

```

```

SELECT
  schemaname,
  relname,
  n_live_tup,
  n_dead_tup,
  round(
    100.0 * n_dead_tup /
    NULLIF(n_live_tup + n_dead_tup, 0),
    2
  ) AS estimated_dead_pct
FROM pg_stat_user_tables
ORDER BY estimated_dead_pct DESC NULLS LAST;

```

The dead-tuple percentage is a screening metric, not a VACUUM FULL trigger. Combine it with physical size, index size, query buffer usage, transaction age and actual latency.

28. Common Myths and Corrections

Myth	Correction
Every UPDATE adds another link every SELECT must walk.	Ordinary sequential scans evaluate heap tuples and visibility; the cost is not simply proportional to a row's lifetime update count.
Dead tuples always mean slow queries.	Dead tuples can contribute to larger relations and I/O, but performance must be proven with plans, buffers and physical size.
VACUUM shrinks the table file.	Normal VACUUM mostly makes space reusable inside PostgreSQL.
VACUUM FULL should run regularly.	It is a table rewrite with ACCESS EXCLUSIVE locking; use only when justified.
HOT means the row is updated in place.	HOT still creates a new heap tuple version.
Changing an unindexed column guarantees HOT.	HOT also requires enough free space on the same page and other eligibility conditions.
Autovacuum should be disabled before bloat testing in production.	Keep autovacuum enabled in production; controlled experiments belong in isolated labs.

29. Real-Time Production Scenario

Assume an online rental platform receives millions of rental-status changes each day. The DBA sees table growth, more dead tuples and larger indexes. The wrong conclusion is that every SELECT is slow because every logical row has many historical versions.

1. Capture the slow query with EXPLAIN (ANALYZE, BUFFERS).

2. Compare heap and index sizes against historical baselines.
3. Check `n_dead_tup`, `n_tup_upd` and `n_tup_hot_upd`.
4. Use `pgstattuple` on targeted suspect relations where the inspection cost is acceptable.
5. Check whether index-only scans show high Heap Fetches.
6. Inspect all-visible page coverage using `pg_visibility`.
7. Check autovacuum history and table-specific storage parameters.
8. Find long-running transactions and idle-in-transaction sessions.
9. Check whether frequently updated columns are over-indexed.
10. Choose maintenance only after the measured bottleneck is clear.

30. DBA Decision Matrix

Finding	Likely Meaning	Preferred DBA Action
High <code>n_dead_tup</code> but stable relation size and good plans	Churn exists; space may be reused effectively	Monitor; confirm autovacuum and latency.
Large heap growth + more buffers per sequential scan	Storage/read amplification likely	Tune vacuum, workload, fillfactor; investigate long transactions.
Low HOT ratio on unindexed-column workload	Insufficient same-page free space or workload/layout issue	Test fillfactor and page-space behavior.
Index-only scan with high Heap Fetches	Pages are not all-visible	Check recent DML and vacuum/visibility-map state.
Dead tuples remain high despite vacuum attempts	Cleanup horizon may be blocked	Inspect long transactions, replication slots/other retention factors as applicable.
Physical file must be returned to OS immediately	Normal VACUUM will usually not be enough	Plan a rewrite strategy; VACUUM FULL only if locking/disk requirements are acceptable.

31. End-to-End Lab Checklist

11. Confirm `dvdrental` is online.
12. Create `mvcc_lab` and supplied diagnostic extensions.
13. Copy `public.rental` to `mvcc_lab.rental_activity`.
14. Add primary key and test indexes.
15. Record initial row count and relation sizes.
16. Record `pg_stat_user_tables` baseline.
17. Inspect `xmin`, `xmax` and `ctid`.
18. Perform an unindexed-column UPDATE.
19. Measure HOT counters.
20. Perform an indexed-column UPDATE.
21. Generate repeated update churn.
22. Measure `n_dead_tup` and `pgstattuple`.
23. Compare heap/index size with baseline.
24. Run sequential-scan EXPLAIN (ANALYZE, BUFFERS).
25. Run index-scan EXPLAIN.
26. Create covering index.
27. VACUUM ANALYZE and run index-only scan test.
28. Record Heap Fetches.
29. Modify rows, repeat index-only scan, then VACUUM and compare again.

30. Inspect the visibility map.
31. Test transactional DELETE + ROLLBACK.
32. Inspect long-running transactions.
33. Review autovacuum settings.
34. Run normal VACUUM and remeasure physical statistics.
35. Optionally run VACUUM FULL only in the isolated lab and compare file size.
36. Document conclusions based on measured pages, buffers, dead tuples, visibility and plan behavior.

32. Cleanup

```
DROP SCHEMA mvcc_lab CASCADE;
```

The original DVD Rental tables remain unchanged because all destructive tests were performed in the isolated mvcc_lab schema.

33. Final Technical Conclusions

- PostgreSQL MVCC stores new row versions for UPDATE and retains obsolete versions until safe cleanup.
- Bloat can increase disk footprint, memory pressure and I/O because the heap or indexes may occupy more pages.
- Bloat is not equivalent to an unlimited version-chain walk for every SELECT.
- HOT reduces index-update overhead when indexed columns are unchanged and enough same-page space exists.
- Intermediate obsolete HOT versions can be pruned during normal operation when safe.
- Normal index scans generally require heap visibility checks; index-only scans can skip heap access when the visibility map proves a page is all-visible.
- VACUUM remains essential for reclaiming reusable space, cleaning indexes, maintaining the visibility map and protecting against transaction ID wraparound.
- Normal VACUUM usually does not compact the relation back to a small operating-system file.
- VACUUM FULL can compact the table, but it rewrites the relation and takes an ACCESS EXCLUSIVE lock.
- DBAs should prove the performance effect with EXPLAIN (ANALYZE, BUFFERS), physical sizes, visibility, HOT counters and transaction-age evidence before choosing corrective maintenance.

Best DBA question: What measurable work is the query doing because of physical relation state — not simply how many dead tuples exist?

34. PostgreSQL Official Documentation References

PostgreSQL 18 - Routine Vacuuming

<https://www.postgresql.org/docs/18/routine-vacuuming.html>

PostgreSQL 18 - Heap-Only Tuples (HOT)

<https://www.postgresql.org/docs/18/storage-hot.html>

PostgreSQL 18 - Visibility Map

<https://www.postgresql.org/docs/18/storage-vm.html>

PostgreSQL 18 - Index Scanning

<https://www.postgresql.org/docs/18/index-scanning.html>

PostgreSQL 18 - Index-Only Scans and Covering Indexes

<https://www.postgresql.org/docs/18/indexes-index-only-scans.html>

PostgreSQL 18 - pgstattuple

<https://www.postgresql.org/docs/18/pgstattuple.html>

PostgreSQL 18 - pg_visibility

<https://www.postgresql.org/docs/18/pgvisibility.html>

PostgreSQL 18 - pageinspect

<https://www.postgresql.org/docs/18/pageinspect.html>

PostgreSQL 18 - Database Physical Storage

<https://www.postgresql.org/docs/18/storage.html>

PostgreSQL 18 - Statistics Collector / pg_stat_* views

<https://www.postgresql.org/docs/18/monitoring-stats.html>

END OF TECHNICAL SOP